VAEs vs GANs vs Diffusion Models

Francisco Tavares, 2020211848
Rodrigo Matias, 2022228254

April 26, 2026

1 Introduction

In this report, we explore the fundamentals of training different generative models. We describe how each model is implemented and the rationale behind it, and we analyze the generated images to discuss each method's strengths and weaknesses. Finally, we propose modifications that could improve model performance.

2 Problem Description

The goal of this project is to train different generative models to produce samples that closely match the data distribution of images of artworks.

The dataset used in this project is ArtBench-10, which contains around 50,000 training images of size 32×32 in RGB. The models must learn to generate images of paintings spanning multiple artistic styles; ArtBench-10 is divided into 10 styles.

Our main task is to compare different families of generative models in terms of training stability, sample quality, and computational cost.

3 Methodology and Approach

This section summarizes the core ideas behind the three generative-model families implemented in this project: Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs), and diffusion models.

3.1 Variational Autoencoders (VAEs)

The general idea of an Autoencoder (AE) compressed data to a lower-representation of the data, and then reconstruct it back to its original form.

However, the classical AEs have shown to have some limitations that prevent the model from giving a quality sample:

- Discontinuous Latent Space: The encoder does not guarantee a smooth latent manifold, since points between two encoded training does not always decode meaningful outputs.
- No Generative Prior: There is no defined distribution over the latent space, so the latent space is not regularized.
- No Interpolation Guarantee: It is possible to have regions of the latent space that decode to garbage which is not suitable for creative generation.
- Latent Space Holes: Small variations can lead to wildly different outputs due to the encoder mapping very different inputs to the same latent code.

The VAEs resolve the biggest issue that the AEs face, the fact that they do not know the exact form of latent distribution, which makes all computations intractable. The way VAEs overcome this obstacle is by organizing the latent space, assuming that the latent distribution is a normal distribution.

Lastly, the decoder takes a random sample from the latent space, where it tries to reconstruct the original image, resulting in new data that are similar to the original data.

3.2 GANs

The idea of a DCGAN model is similar to a min-max game, where in this generative model there will be two neural networks that "compete" with each other:

- Discriminator: responsible, in the context of this project, for judging if the received image is considered generated or is a real image.
- Generator: responsible to trick the Discriminator into thinking that a generated image is real.

With those two networks, DCGAN models go through this process:

3.2.1 Generator's First Move

The first ever image that the Generator will produce is a noise vector with random values. This noise is the fake data that, alongside the real data, will be sent to the Discriminator.

3.2.2 Discriminator's Turn

As said in the previous step, the Discriminator receives two types of data:

- Real samples from the dataset.
- Fake samples from the Generator.

For each sample, the Discriminator analyzes their input. The output of this step is a probability score between 0 and 1 that shows how confident the Discriminator is in thinking the sample is real or not, where 1 says the data are real and 0 suggests it is fake.

3.2.3 Adversarial Learning

Every conclusion that the Discriminator makes will determent how much positive and negative changes each network will receive. If it guesses correctly the type of data, the standard will be higher for the Generator to trick it. In case the Generator successfully fools the Discriminator, it receives a better fitness, while the Discriminator will be penalized for its mistake.

3.2.4 Generator's Improvement and Discriminator's Improvement

Both networks, throughout many iterations, get better over time. By this constant back-and-forth, the Generator will be able to create more convincing fake samples where as the Discriminator will be better at spotting fake data

3.2.5 Training Progression

In theory, as training continues, the Generator is able to produce such quality samples that the Discriminator is never going to suspect that their samples were generated. At this point, it has been concluded that the DCGAN models can produce images very similar to realistic photos.

3.3 Diffusion

The main idea of Diffusion models is to be able to recreate a recognizable image from a sample of pure noise. In order for that to be possible, the model need to complete these steps:

3.3.1 Forward Diffusion Process

For every sample in the dataset, they go through a Markov chain where they only add Gaussian noise, it does not involve any learning. This process is not finished until the data become images of pure noise.

3.3.2 Reverse Process (Learning to Denoise)

In this step, it is the opposite of the Forward Distribution Process, where the model learns how to reverse the noise previously added by denoising the data to produce a similar structure as the original dataset.

3.3.3 Training Objective (Loss Function)

What training does is predict the noise added at each step by comparing their prediction with the actual noise; therefore, the model demonstrates having a simple way of learning as they solve a noise prediction problem rather than having a complex generation task.

3.3.4 Sampling and Data Generation

After finishing training, the model generates new random noise, testing their ability to form an understanding image. It is expected:

- It can apply denoising steps until it can form a realistic sample.
- Each step will remove a small amount of noise, resulting in the model being able to create patterns.
- More steps will produce more detail images. On the other hand, fewer steps result in less detailed samples.

4 Experimental Setup

This experiment compares the performance of VAE, DCGAN, and Diffusion models under different training configurations in order to evaluate their image generation capabilities, training stability, and output quality. The selected hyperparameters were chosen based on commonly used values in generative deep learning tasks to balance convergence speed, training stability, and computational efficiency they were later fine-tuned using grid search to accommodate smaller epoch size while still maximizing results. Values were fined tuned using only a subset of the full dataset corresponding to 20% which is around 10000 images

The models were evaluated using FID and KID metrics, which measure the similarity between generated images and real dataset samples. Multiple repetitions were performed to ensure reliable evaluation and to observe the consistency of each model during training.

Adam was the optimizer of choice for all the models.

4.1 VAE setup

The VAE model was trained using the following hyperparameters:

- Learning Rate = 1×10^{-3}
- Beta = 0.35 (beta is multiplied by kl divergence loss do reduce or increase the importance of achieving a normal distribution)
- Epochs = 50

- Latent Space = 128
- Has a relative similar network siz

A learning rate of 1×10^{-3} was selected to allow faster convergence while maintaining stable optimization during training. The beta parameter was set to 0.35 to reduce excessive latent regularization and improve reconstruction quality. The model was trained for 50 epochs to provide sufficient learning iterations for stable latent representation learning and image reconstruction.

Warm up phase. We tested introducing a warm up period of 5 epochs where beta is but a fraction of it's value and increases proportional to the epoch we are in currently. This allowed the model to learn how to reconstruct images really well and only then begin to gradually learn how to approximate a normal distribution. This changed allowed for a higher beta value to be introduced while still using only 50 epochs for training.

4.2 DCGAN setup

The VAE model was trained using the following hyperparameters:

- Learning Rate = 2×10^{-4}
- Epochs = 50
- Latent Space = 128

A smaller learning rate was used for DCGAN training because adversarial optimization is highly sensitive to parameter updates. Using 2×10^{-4} helps improve training stability and reduces oscillation between the generator and discriminator. The model was trained for 50 epochs to balance computational cost and generated image quality.

4.3 Diffusion setup

The Diffusion model was trained using the following hyperparameters:

- Learning Rate = 2×10^{-4}
- Epochs = 50

The selected learning rate provides stable optimization during the denoising learning process while avoiding unstable parameter updates. Training for 50 epochs allowed the model to progressively learn image structures and improve generation quality while maintaining reasonable training time.

Overall, the selected hyperparameters were designed to provide fair experimental conditions for comparing the three generative approaches in terms of image quality, stability, and consistency across multiple training repetitions.

5 Analysis of Results

We ended up only validating the results for 1000 images and only with 3 repetitions because the estimated time to perform this task was 30+ hours. Unfortunately, we did not foresee this problem and only found around 24 hours before the deadline. So this change was necessary in order to get results before the deadline.

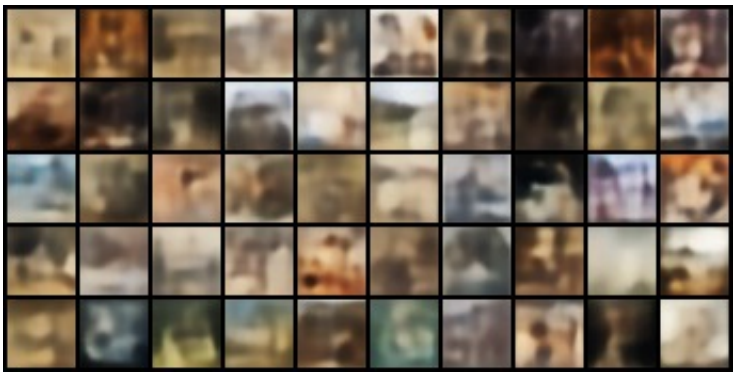

Figure 1: Random VAEs generated images

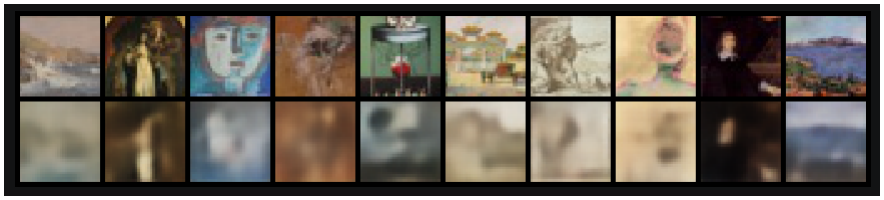

Figure 2: Original images vs VAEs generated images

[VAE] FINAL STATISTICS (over 3 repetitions):

- FID: 206.4603 ± 1.6811
- KID: 0.1705 ± 0.0023

The Fréchet Inception Distance (FID) obtained a mean value of 206.4603 with a standard deviation of 1.6811. The mean FID score indicates that the generated images are still significantly different from the real dataset distribution, which is expected for VAE models since they generally produce blurrier outputs and struggle to preserve fine details. However, the relatively low standard deviation shows that the model produced consistent results across different

runs, meaning that the training process was stable and the quality of generated images did not vary significantly between repetitions.

Similarly, the Kernel Inception Distance (KID) achieved a mean value of 0.1705 with a standard deviation of 0.0023. Since lower KID values indicate better similarity between generated and real images, the obtained result suggests that the VAE was able to learn general image structures and patterns, although the generated samples still differ noticeably from the original dataset. The very small standard deviation further indicates that the model performance remained highly consistent across multiple executions.

Overall, the low standard deviations for both FID and KID demonstrate that the VAE model has stable training behavior, while the relatively high mean scores indicate limitations in generating highly detailed and realistic images.

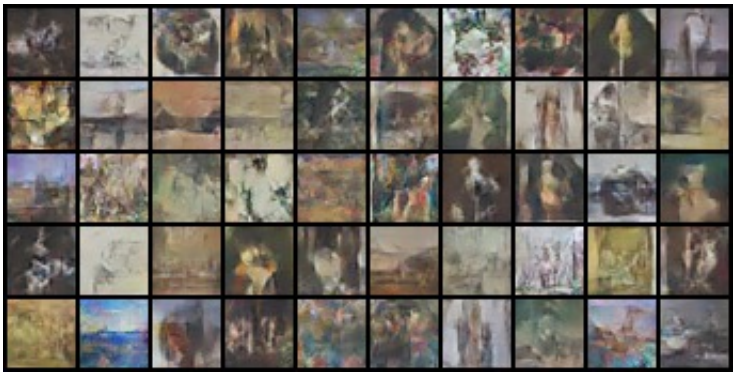

Figure 3: GANs generated images result

[DCGAN] FINAL STATISTICS (over 3 repetitions):

- FID: 109.2592 ± 2.0624
- KID: 0.0598 ± 0.0033

The Fréchet Inception Distance (FID) achieved a mean value of 109.2592 with a standard deviation of 2.0624. Compared to the VAE model, the lower FID score indicates that the images generated by the DCGAN are significantly closer to the real dataset distribution and contain better visual quality and sharper details. This demonstrates the capability of adversarial training to produce more realistic samples. The relatively small standard deviation also suggests that the model maintained stable performance across multiple runs, although slight variations are expected due to the stochastic nature of DCGAN training.

Similarly, the Kernel Inception Distance (KID) obtained a mean value of 0.0598 with a standard deviation of 0.0033. The lower KID value indicates

improved similarity between generated and real images when compared to the VAE model. This suggests that the DCGAN was more effective in learning the underlying image distribution and reproducing more realistic structures and textures. The small standard deviation further shows that the model produced consistent results across different training repetitions.

Overall, the lower mean FID and KID scores demonstrate that the DCGAN model generated higher-quality images than the VAE model. Although DCGAN training can be unstable, the relatively low standard deviations indicate that the selected training configuration provided reasonably stable and reproducible results.

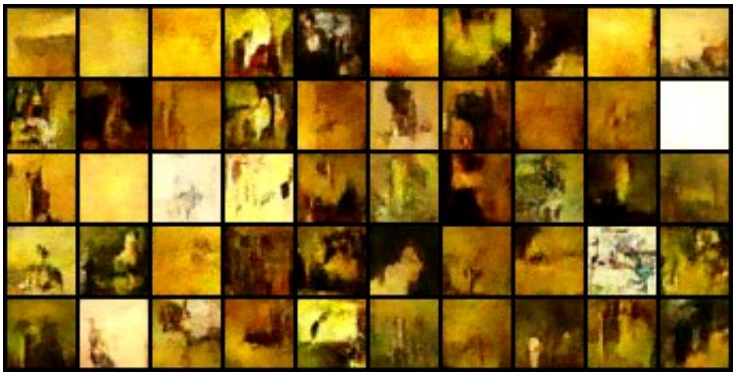

Figure 4: Diffusion generated images result

[Diffusion] FINAL STATISTICS (over 3 repetitions):
FID: 215.2049 ± 2.0015
KID: 0.1800 ± 0.0018

[Diffusion] FINAL STATISTICS (over 3 repetitions):

- FID: 215.2049 ± 2.0015
- KID: 0.1800 ± 0.0018

So after looking at the picture you probably already noticed. But we made a mistake. We used batch normalization in our diffusion model image generation, which caused all the image generated in a batch to share colors between each other. We only noticed this late in the validation phase because in training we where checking with very small batches of 10 pictures at a time. And so we get this funny mistake that severely affects the diffusion model's results. The patterns it generates are probably still the best out of all the models, but this color issue puts it behind VAE in score.

The Fréchet Inception Distance (FID) achieved a mean value of 215.2049 with a standard deviation of 2.0015. This relatively high FID score indicates

that the generated images still differ considerably from the real dataset distribution. Compared to the DCGAN model, the Diffusion model produced lower-quality outputs under the selected training configuration. One possible reason for this result is that Diffusion models generally require longer training time, larger datasets, and more computational resources to fully learn the denoising process and generate highly detailed images. Despite the high mean FID value, the low standard deviation shows that the model performance remained stable across multiple repetitions.

Similarly, the Kernel Inception Distance (KID) obtained a mean value of 0.1800 with a standard deviation of 0.0018. The higher KID score suggests that the generated samples still have noticeable differences from the original dataset images and that the model struggled to accurately reproduce fine image structures and textures. However, the very small standard deviation indicates that the generated results were highly consistent across different runs, demonstrating stable training behavior.

Overall, while the Diffusion model showed stable and reproducible performance, the relatively high FID and KID scores indicate that the selected training setup was insufficient for the model to achieve high-quality image generation. Additional training epochs, improved hyperparameter tuning, or greater computational resources may be required to fully utilize the capabilities of the Diffusion model.

6 Conclusion

After receiving the results of each model, the DCGAN model performed the best in terms of image generation, having the lowest FID and a low KID value, meaning that DCGAN was able to produce samples with the most similar structure to the dataset consistently.

This was an unexpected result because, in theory, Diffusion models should have generated sharper samples, since their strengths are having high quality samples with good diversity at the cost of being computationally heavy. However, as previously said, we made a mistake that affected the FID and KID value of the Diffusion model and, as a result, influenced the final conclusion of the project.

7 Sources and AI Tools Used

- https://www.geeksforgeeks.org/machine-learning/variational-autoencoders/
- https://www.geeksforgeeks.org/deep-learning/generative-adversarial-network-gan/
- https://www.geeksforgeeks.org/artificial-intelligence/what-are-diffusion-models/
- ChatGPT-5.2 was used to make this report

- Gemeni 3.1 Pro was used for otimization of code and batch image genaration functions. Also helped generate the validation pipeline that calculate KID and FID (Although the batch normalization issue was not cause by it. It was entirely our fault. And the issue is not in this function but in the model it self.)
- https://www.youtube.com/watch?v=qJeaCHQ1k2w (help understanding VAE)
- https://www.youtube.com/watch?v=RAa55G-oEuk (help understanding GAN)
- https://www.youtube.com/watch?v=fbLgFrlTnGU (help understanding Diffusion)
- https://www.youtube.com/watch?v=a4Yfz2FxXiY